Count Pairs Divisible by K (HashMap + Remainder Pattern)

Problem Statement

Given an integer array `arr` and an integer `k`, count the number of pairs `(i, j)` such that

- `i < j`
- `(arr[i] + arr[j]) % k == 0`

Example

```
arr = [2,2,1,7,5,3]
k = 4
```

Output

```
5
```

Pairs are

```
(2,2)
(1,7)
(1,3)
(7,5)
(5,3)
```

Observations

The condition is

```
(arr[i] + arr[j]) % k == 0
```

Instead of thinking about the original numbers, think about their **remainders** after dividing by `k`.

Example

```
k = 4

Number      Remainder
---------------------
2              2
2              2
1              1
7              3
5              1
3              3
```

Now the array becomes

```
2 2 1 3 1 3
```

We only care about these remainders.

Step 1 — Brute Force

Check every pair.

```python
def bruteForce(arr, k):
    n = len(arr)
    ans = 0

    for i in range(n):
        for j in range(i + 1, n):

            if (arr[i] + arr[j]) % k == 0:
                ans += 1

    return ans
```

Time Complexity

```
O(n²)
```

Space

```
O(1)
```

Why Brute Force is Slow

For every element

```
search every remaining element
```

Example

```
2

check with

2
1
7
5
3
```

Then

```
2

check with

1
7
5
3
```

Too many comparisons.

Step 2 — Build the Logic

We need

```
(a+b)%k==0
```

Suppose

```
a % k = r
```

Need another remainder

```
x
```

such that

```
(r+x)%k==0
```

Move `r`

```
x = k-r
```

But remainder should always stay inside

```
0...k-1
```

Hence

```
need = (k-r)%k
```

This is the formula to remember.

Step 3 — Build Complement Table

Example

```
k = 4
```

Possible remainders

```
0
1
```

```
2
3
```

Need

Current remainder	Need
0	0
1	3
2	2
3	1

Notice

```
1 pairs with 3

2 pairs with 2

0 pairs with 0
```

General formula

```
need = (k-r)%k
```

Main Idea

Instead of searching every future element,

store the frequency of previous remainders.

Whenever a new number arrives

```
find its remainder

↓

find the complementary remainder

↓

how many times have we already seen it?

↓
```

```
add to answer

↓

store current remainder
```

Dry Run

Input

```
arr=[2,2,1,7,5,3]

k=4
```

Initially

```
mp={}

ans=0
```

Element = 2

```
r = 2

need = 2
```

Previous 2s

```
0
```

Answer

```
0
```

Store

```
mp

2 : 1
```

Element = 2

```
r=2

need=2
```

Previous

```
2 : 1
```

Found

```
1 pair
```

Answer

```
1
```

Store

```
2 : 2
```

Element = 1

```
r=1

need=3
```

Previous

```
none
```

Store

```
1 : 1

2 : 2
```

Element = 7

```
r=3

need=1
```

Previous

```
1 : 1
```

Found

```
1 pair
```

Answer

```
2
```

Store

```
1 : 1

2 : 2

3 : 1
```

Element = 5

```
r=1

need=3
```

Previous

```
3 : 1
```

Found

```
1 pair
```

Answer

```
3
```

Store

```
1 : 2

2 : 2

3 : 1
```

Element = 3

```
r=3

need=1
```

Previous

```
1 : 2
```

Found

```
2 pairs
```

Final

```
Answer = 5
```

Why Do We Check Before Storing?

Wrong

```
store

↓

check
```

Suppose

```
2
```

Store first

```
2 : 1
```

Then search

```
need=2
```

You pair the element with itself.

Wrong.

Correct order

```
Find remainder

↓
```

```
Find complement

↓

Count previous complements

↓

Store current remainder
```

Exactly the same idea used in Prefix Sum HashMap problems.

Building the Code Step-by-Step

Step 1

```
mp={}
ans=0
```

Step 2

Loop

```
for num in arr:
```

Step 3

Find remainder

```
r=num%k
```

Step 4

Find complement

```
need=(k-r)%k
```

Step 5

Count previous complements

```
ans+=mp.get(need,0)
```

Step 6

Store current remainder

```
mp[r]=mp.get(r,0)+1
```

Done.

Final Code

```
def optimized(arr, k):

    mp = {}
    ans = 0

    for num in arr:

        r = num % k

        need = (k - r) % k

        ans += mp.get(need, 0)

        mp[r] = mp.get(r, 0) + 1

    return ans
```

Complexity

Time

```
O(n)
```

Space

```
O(k)
```

Maximum distinct remainders are only

```
0...k-1
```

Pattern Recognition

Whenever you see

```
(a+b)%k==0

(a+b)%m==0

pair divisible

pair remainder

pair modulo
```

Immediately think

```
Store remainder frequencies.
```

General Pattern

Instead of

```
Searching future elements
```

Transform each element into a key

```
number

↓
```

```
remainder

↓

frequency
```

Then search only the complementary key.

Pattern Template

```
for every element

    transform into key

    find complementary key

    answer += frequency[complement]

    store current key
```

Generic Code Template

```
mp={}

answer=0

for value in array:

    key = transform(value)

    complement = getComplement(key)

    answer += mp.get(complement,0)

    mp[key]=mp.get(key,0)+1
```

Only

```
transform()

and
```

```
getComplement()
```

change from problem to problem.

Similar Interview Problems

1. Two Sum

Store

```
number
```

Need

```
target-number
```

Pattern

```
HashMap Complement
```

---

2. Count Pairs Divisible By K

Store

```
remainder
```

Need

```
(k-r)%k
```

Pattern

```
Remainder Complement
```

---

3. Subarray Sum Equals K (LC 560)

Store

```
prefix sum
```

Need

```
prefix-k
```

Pattern

```
Prefix Sum HashMap
```

---

4. Continuous Subarray Sum (LC 523)

Store

```
prefix remainder
```

Need

```
same remainder
```

Pattern

```
Modulo Prefix Sum
```

---

5. Subarrays Divisible By K (LC 974)

Store

```
prefix remainder
```

Need

```
same remainder
```

Pattern

```
Modulo Prefix Sum
```

---

6. Count Nice Pairs

Store transformed values.

Pattern

```
HashMap Frequency
```

---

7. Count Bad Pairs

Again

```
Transform
↓
Frequency
↓
Count
```

Same family.

How to Think During Interviews

Suppose interviewer asks

```
Count pairs...
```

Ask yourself

Question 1

Can I compare every pair?

```
Yes

↓

O(n²)
```

Can it be improved?

Question 2

Can every element be represented using a smaller key?

Examples

```
remainder

prefix sum

difference

parity

frequency
```

Question 3

If current key is

```
x
```

What previous key do I need?

This is called

```
Complement
```

Question 4

Can I store frequencies?

If yes

```
HashMap
```

Recognition Checklist

Whenever you see

✅ Count pairs

✅ Count frequencies

✅ Divisible

✅ Modulo

✅ Sum equals something

✅ Complement exists

Think

```
HashMap
```

Then ask

```
What should be my key?
```

Real World Analogy

Imagine

```
Bus seats numbered

0

1

2
```

```
3
```

A rule says

```
Seat numbers must together make a multiple of 4.
```

A passenger arrives at seat

```
1
```

He doesn't scan every previous passenger.

He immediately asks

```
How many people are sitting in seat remainder 3?
```

Those are exactly his partners.

HashMap behaves like this register.

Common Mistakes

❌ Forgetting modulo

Wrong

```
need = k-r
```

Correct

```
need=(k-r)%k
```

❌ Store before checking

Wrong

```
Store

↓

Check
```

Self pairing happens.

❌ Using original numbers instead of remainders

Store

```
num % k
```

Not

```
num
```

Quick Revision Sheet

Formula

```
remainder = num % k

need = (k-remainder)%k
```

Algorithm

```
for every number

    find remainder

    compute complement

    answer += frequency[complement]

    frequency[current]++
```

```
return answer
```

Complexity

```
Time : O(n)

Space : O(k)
```

Recognition Keywords

```
Pairs

Divisible

Modulo

Complement

HashMap

Frequency

Remainder
```

Master Pattern

```
Brute Force

For every element

↓

Search every other element

↓

Can I convert each element into a key?

↓

Can I compute the complementary key?
```

```
↓
Store previous key frequencies
↓
Lookup complement in O(1)
↓
Store current key
↓
Overall O(n)
```

One-Line Memory Trick

> **"Whenever a pair condition can be rewritten as 'current key + complementary key', store frequencies of the key in a HashMap and look up the complement instead of searching all previous elements."**